

ProvenanceRank — Documentation

A complete, beginner-friendly guide to the project: what it does, how it's built, the technology behind every feature, and how you could rebuild it from scratch.

Read the files in order if you're new. Jump straight to **04** if you just want to know what a particular file does.

PDF versions are provided too: a single combined [ProvenanceRank_Documentation.pdf](#) (the whole thing, with a table of contents), and a per-file PDF for each document under [pdf/](#). The Markdown files are the source of truth.

#	File	What's inside
00	this file	Index + how to read the docs
01	01_overview.md	The problem, what the system does, the full tech stack, repo map
02	02_architecture.md	The two-phase design, data flow, how the layers fit, diagrams
03	03_features.md	Every feature, the tech behind it, and how it works
04	04_file_by_file.md	A reference for every module and file in the repo
05	05_build_from_scratch.md	Step-by-step: rebuild the project yourself, in order
06	06_glossary.md	Plain-English definitions: NDCG, BM25, RRF, LambdaMART, etc.
07	07_running_guide.md	How to run every section — engine, tests, API, UI, live layer, graph RAG, Docker, with copy-paste commands

The 30-second version

ProvenanceRank takes a **job description** and a file of **100,000 candidate profiles** and produces a ranked **top-100** shortlist (`submission.csv`), each row with a one-line justification. It was built for a hiring-AI hackathon (rank a “Senior AI Engineer — Founding Team” role) and then grown into a production-style service with an API, database, authentication, observability, a React UI, and a “live” career-intelligence layer that ingests real GitHub activity into a verifiable proof-of-work graph.

The graded part runs under hard constraints: **no network, CPU only, under 5 minutes, exactly 100 rows**. The trick that makes that possible is a **two-phase split** — all the slow, smart work happens *offline* and is cached; the graded step just reads the cache. See [02_architecture.md](#).

Other docs in the repo

These pre-date this folder and go deeper on specific areas:

- [../README.md](#) — top-level project readme
- [../RUN_LOCAL.md](#) — how to run it on your laptop (3 ways)
- [../ACCURACY.md](#) — the accuracy + performance layers in depth
- [../SYSTEM_DESIGN.md](#) — production system-design write-up
- [../ARCHITECTURE.md](#) — architecture diagram + rationale

Conventions used in these docs

- **“Offline”** = the `precompute.py` phase. Allowed to be slow and use the network.
- **“Online”** / **“graded”** = the `rank.py` phase. No network, CPU, < 5 min.
- **“Graceful degradation”** = every heavy dependency (torch, xgboost, Redis, Neo4j, Postgres, a GPU, an API key) is *optional*. If it's missing, the code falls back to a pure-Python path and still runs. This is a core design rule, so you'll see it mentioned a lot.